Performance Testing Report Submission Template

Computer Systems and Network Engineering - Final Year Research Project

Project Title: **Lightweight Application-Layer IDS for Simultaneous Sinkhole and Selective Forwarding Detection in RPL Networks**

Student Names: **Wijetunga S. S.**

Student IDs: **IT22342126**

Supervisor: **Ms. Shashika Lokuliyana, Mr. Anuradha Jayakody**

Date of Submission: **12th April 2026**

1. Introduction and Objective of Performance Testing

1a. Purpose and Scope

This report outlines the design, implementation, and results of the performance analysis of the Lightweight Application-Layer Intrusion Detection System (IDS) - Component 2 of a four-component integrated security framework for RPL-based IoT networks. The IDS detects Sinkhole and Selective Forwarding Attack (SFA) simultaneously, using three passive behavioral metrics: Packet Timing Interval (PTI), Sequence Integrity Score (SIS), and Control/Data Forwarding Ratio (CDFR); assessed over a 10-second sliding window at each individual node. Performance Testing captures to what degree the system satisfies its primary design objective of zero false positives (FP=0) under both static and mobile IoT scenarios, alongside increasing true positive (TP) counts as attacks continue.

1b. Types of Performance Testing Conducted

Testing Type	Mapped To IDS Context	Justification
Functional Detection Testing	Validates TP/FP/FN/TN accuracy per 10-second window under simultaneous Sinkhole + SFA attack	Core correctness test: confirms the IDS correctly classifies attack vs. normal traffic
Stress Testing	Tests IDS under mobile topology: 20+ parent changes, ETX=7.44 (severely degraded wireless links)	Validates robustness under worst-case dynamic network conditions beyond normal operating range
Endurance Testing	Runs IDS continuously across 3–5 minutes with attacks active to observe TP growth stability	Confirms sustained detection accuracy does not degrade over time; no memory leaks or counter overflow
Scalability / Load Testing	Validates distributed, per-node IDS operation across all 6 nodes simultaneously without coordination	Confirms architecture scales without centralized bottleneck: each node processes its own detection workload
Baseline / Regression Testing	Attacks disabled (ENABLE flags=0); validates TN accumulation and FP≈0 during normal RPL operation	Ensures IDS does not generate false alarms under routine RPL traffic: critical for energy efficiency

1c. Performance Benchmarks and Expected Outcomes

Metric	Benchmark Target	Rationale
False Positive Rate (FP)	FP = 0 in ALL windows (static & mobile)	Spurious alerts drain energy and trigger unwarranted isolation on constrained IoT nodes
True Positive Growth	TP grows each window from W1 onward	Confirms progressive, sustained attack detection: not a single lucky detection
Simultaneous Attack Detection	Severity ≥ 2 by Window 3 (t=30s)	PTI + CDFR compound classification must fire within 30 seconds of attack launch

PTI Threshold (Static)	704 ticks - 4,266 ticks (adapts)	Threshold must learn normal traffic within 3 windows and stabilize above normal PTI
PTI Threshold (Mobile)	6,000-8,000 ticks (wider)	Threshold must widen in mobile to absorb link variance while keeping FP=0
Packet Delivery Ratio (PDR)	Drops from ~1.00 to 0.30–0.40 under attack	Quantifies attack damage; confirms Sinkhole + SFA compound impact on network
RAM Footprint	< 10 KB (Sky mote limit)	Confirms lightweight constraint: IDS must fit within constrained node memory

2. Performance Testing Environment Setup

2a. Hardware Configuration

Component	Specification	Justification
Host / Server Machine	Windows 11 + WSL2 (Ubuntu 24.04 LTS), multi-core x86-64, ≥8 GB RAM	Hosts Cooja JVM simulator and Contiki-NG GCC toolchain; WSL2 provides native Linux environment on Windows
Emulated Sensor Nodes	6 × Cooja Sky motes - MSP430 @ 8 MHz, 10 KB RAM, 48 KB Flash	MSP430-based Sky mote matches IEEE 802.15.4 IoT hardware profile used in benchmark literature [18]
Network Topology	1 DAG root + 2 attacker nodes (Sinkhole: Node 3, SFA: Node 2) + 1 faulty node (Node 4) + 2 victim/observer nodes (Nodes 5–6)	Consistent with Patel & Shah [18]; places attackers at strategic hops to maximize compound attack impact
Radio Hardware (Emulated)	CC2420 radio (emulated by UDGM), 2.4 GHz, IEEE 802.15.4	CC2420 is the standard transceiver on Sky motes; emulation accurately models link-layer behaviour

2b. Software Configuration

Software / Tool	Version	OS / Platform	Purpose / Test Scripts
Contiki-NG	v4.9+ (GitHub)	Ubuntu 24.04 (WSL2)	IoT OS; RPL stack, etimer, UDP APIs. IDS implemented in udp-client.c; attack in rpl-mrhof.c
Cooja Network Simulator	Bundled w/ Contiki-NG	Java 17 / Ubuntu	Mote emulation; IDS_Static.csc and IDS_Mobile.csc simulation scripts; real-time Mote Output logs
GCC / MSP430 Toolchain	gcc-msp430 (APT)	Ubuntu 24.04	Cross-compiles IDS firmware for Sky mote target; validates 10 KB RAM constraint
attack_config.h	Custom (project)	Compile-time flags	Test script: sets SINKHOLE_NODE_ID=3, SFA_NODE_ID=2, DROP_PERCENT=30, ENABLE_SINKHOLE_ATTACK, ENABLE_SELECTIVE_FORWARDING

Python 3 + matplotlib	3.12+ / 3.7+	Ubuntu 24.04	Post-test analysis: parses Cooja log output; generates metric charts (Figures 1–3 in this report)
Git	2.43+	Ubuntu 24.04	Version control for .csc simulation files and IDS source: ensures reproducibility across test runs

2c. Network Conditions

Network Parameter	Static Simulation	Mobile Simulation	Relevance to IDS Testing
Network Bandwidth	IEEE 802.15.4, ~250 kbps	IEEE 802.15.4, ~250 kbps (degraded by high ETX)	Sets baseline packet rate context for PTI and CDFR metric calibration
Simulated Packet Loss	SFA node drops 30% of packets (probabilistic, P(drop)=0.30)	SFA 30% drop + natural mobility-induced loss	Tests IDS sensitivity to intentional vs. benign loss; SIS threshold calibrated against this
Latency (normal)	PTI: 1,000–2,000 ms inter-packet	PTI variable; avg ETX=7.44 inflates delay	PTI baseline for dynamic threshold learning (target: threshold stabilizes at ~4,266 ticks)
Latency (under attack)	15,000+ ticks (Sinkhole rerouting through sub-optimal path)	15,000+ ticks (still detectable above widened 6,000–8,000 tick threshold)	Validates attack PTI spikes remain detectable above adapted threshold in both environments
Parent Changes	1 (initial DODAG join only)	20+ (continuous topology churn)	Stress-tests dynamic threshold: natural delay spikes from parent changes must not trigger FP
Simulation Duration	3–5 minutes; attacks at t=120s	3–5 minutes; attacks at t=120s	Endurance test window; sufficient for 4+ observation windows post-attack launch

Compliance notes: All simulated traffic is synthetic: RPL DIO/DAO control messages and UDP payloads containing only incrementing sequence numbers ('hello N'). No bandwidth is consumed on real networks. No GDPR, HIPAA, or PCI DSS obligations apply.

3. Tools and Frameworks Used

3a–3b. Tools Selected, Configuration, and Rationale

Tool	Configuration & Usage	Why Chosen	Reference
Contiki-NG v4.9+	IDS logic implemented entirely in udp-client.c (application layer). Rank manipulation in rpl-mrhof.c (rank_via_nbr()). No kernel	Production-grade IoT OS; actual RPL stack deployed on real IEEE	github.com/contiki-ng

	modifications — preserves OS integrity and portability.	802.15.4 hardware worldwide. Standard platform for RPL security research [17][18][22].	
Cooja Simulator	6-mote Sky network. Simulation scripts: IDS_Static.csc (fixed topology), IDS_Mobile.csc (positions.dat mobility). Mote Output filters: ALERT, SIMULTANEOUS, TP=, Dynamic PTI, ATTACK SH/SFA.	Gold-standard RPL simulation environment [18][22]; deterministic, reproducible, hardware-accurate emulation without physical hardware requirements.	docs.contiki-ng.org
attack_config.h (Custom Test Script)	Centralised compile-time flags: SINKHOLE_NODE_ID=3, SFA_NODE_ID=2, DROP_PERCENT=30, FAKE_RANK_IMPROVEMENT=3, ENABLE_SINKHOLE_ATTACK, ENABLE_SELECTIVE_FORWARDING. Toggle ON/OFF without code changes.	Single source of truth for all attack parameters; enables controlled attack vs. baseline (FP/TN) testing through simple flag changes: equivalent to test configuration files in JMeter.	Project file
Built-in IDS Counters (Custom)	TP/FP/FN/TN counters in udp-client.c update per window vs. ground truth. PTI circular buffer (size 3). CDFR counters (c_ctrl, c_data). SIS miss counter. Per-window log: [IDS] Node X | Severity=Y PTI=A SIS=B CDFR=C.	Eliminates need for external profiler on 10 KB constrained nodes. Provides per-node per-window precision measurement : equivalent to JMeter's built-in listeners for	Project file

		constrained IoT.	
Python 3 + matplotlib 3.7+	Post-test log parsing and metric chart generation. Produces Figures 1–3 in this report from structured Cooja log output.	Lightweight, reproducible, open-source. Standard tool for IoT simulation result visualization in published research.	matplotlib.org

3c. References to Tool Documentation and Performance Standards

- **Contiki-NG:** Contiki-NG Official Documentation: https://docs.contiki-ng.org: RPL stack, etimer API, Sky mote specifications
- **Cooja:** Cooja Simulator Tutorial: https://docs.contiki-ng.org/en/latest/doc/tutorials/Running-Contiki-NG-in-Cooja.html
- **IEEE 802.15.4:** IEEE 802.15.4-2015 Standard: Low-Rate Wireless Networks (defines the radio layer modelled by the UDGM radio model in Cooja)
- **RPL Standard:** IETF RFC 6550: IPv6 Routing Protocol for Low-Power and Lossy Networks (RPL). Defines DODAG construction, DIO/DAO messages, rank computation all tested by the IDS
- **Performance Standard:** Patel & Shah [18] and Verma & Ranga [22]: Published benchmarks establishing Cooja + Contiki-NG as the accepted performance testing standard for RPL IoT security research

4. Testing Methodology and Execution

4a. Test Scenarios

Scenario	Type	Config	Execution Steps
A - Simultaneous Attack (Static Topology)	Functional Detection	ENABLE_SINKHOLE=1 ENABLE_SFA=1 IDS_Static.csc	1. Compile: make TARGET=cooja 2. Launch Cooja; load IDS_Static.csc 3. Start simulation; attacks auto-trigger at t=120s 4. Filter: SIMULTANEOUS, ALERT, TP= 5. Record TP/FP/FN/TN per 10-s window
B - Simultaneous Attack	Stress Testing	ENABLE_SINKHOLE=1 ENABLE_SFA=1 IDS_Mobile.csc	1. Load IDS_Mobile.csc with positions.dat script 2. Run with same attack

(Mobile Topology)			config 3. Record FP count, parent changes, ETX 4. Verify PTI threshold range = 6,000–8,000 ticks
C - No Attack (Baseline)	Endurance / Regression	ENABLE_SINKHOLE=0 ENABLE_SFA=0 IDS_Static.csc	1. Disable attacks in attack_config.h; rebuild 2. Run 3 minutes; filter TP= 3. Record TN growth and FP count 4. Capture baseline CDFR and PTI values

4b. Test Data — Nature and Volume

Data Element	Nature	Volume / Parameters
UDP Application Payload	Synthetic: 'hello N' string (N = incrementing sequence number). No real user data.	~5 packets per 10-second window per node (at 0.5 Hz send rate). Total: ~30 packets/window across all 6 nodes.
RPL Control Traffic (DIOs)	RPL DIO messages broadcast by each node advertising rank and DODAG membership.	1 DIO per DODAG instance per window (Trickle timer controlled). Sinkhole attack inflates this count.
Attack Traffic - Sinkhole	Fabricated DIO rank values: r_adv = r_comp / K (K=3): makes Node 3 appear 3× closer to root.	Activated at t=120s; affects all nodes within Node 3's radio range. Rank manipulation is a single parameter change.
Attack Traffic - SFA	Probabilistic packets drop at Node 2: P(drop) = D/100, D=30. Silent discard of received UDP packets.	~1.5 packets dropped per 10-second window (30% × 5 pkts). Sequence gaps visible in SIS metric.
IDS Log Output	Structured per-node per-window log lines from Cooja Mote Output.	~24 log entries per window (4 observation windows × 6 nodes). Parsed post-simulation for analysis.

4c. Workload Distribution

The 8 nodes operate as individual independent instances of the IDS, and lack inter-node cooperation during the detection phases. The expected workload per node per 10-second window is: (1) receive a maximum of 5 UDP packets via udp_rx_callback() — each of which triggers a PTI update and SIS assessment ; (2) poll the RPL DAG status exactly once via rpl_get_any_dag() to increment CDFR C_ctrl; (3) at the window boundary, compute the 3 metric flags, execute the

severity classifier, and update the TP/FP/FN/TN counters, write to the log, and reset the counters. This distributed execution framework implies that the detection workload scales linearly with the number of nodes - i.e. the addition of nodes does not create centralized processing bottlenecks. The 10-second shared etimer synchronizes the window boundaries of all six nodes without incurred communication overhead.

5. Metric Measured

Guideline Category	Metric	Formula / Measurement	Normal Value	Under Attack
Response Time	Packet Timing Interval (PTI)	PTI = current_rx_time − last_rx_time (Contiki-NG clock ticks via clock_time()). Flagged when PTI > PTI_threshold.	1,000–2,000 ms (204–409 ticks)	15,000+ ticks (Sinkhole rerouting adds multi-hop delay)
Throughput	Packet Delivery Ratio (PDR)	PDR = packets_received / packets_sent per 10-s window per node. Tracked via win_sent_window and CDFR counters.	1.00 (100%)	0.30–0.40 under combined Sinkhole + SFA [18]
Throughput	Control/Data Fwd Ratio (CDFR)	CDFR = C_ctrl / C_data per window. Flagged if CDFR < 0.5. Tracks imbalance between RPL control and UDP data traffic.	≥ 1.0	0.17 at W3 (data=6, but SFA drops suppress ratio)
Resource Utilisation	RAM / ROM Footprint	Code size within Sky mote 10 KB RAM and 48 KB Flash limit. Verified at compile time via MSP430 GCC size output.	< 10 KB RAM	Compliant - no kernel modifications; no additional RAM allocation under attack
Resource Utilisation	CPU / Processing Overhead	IDS processing per window: 3 metric computations + 1 severity score + log output. No ML inference, no crypto.	< 5% CPU cycles (estimated)	No increase under attack - same fixed O(1) computation per window
Resource Utilisation	Network Bandwidth Overhead	IDS generates zero additional network traffic during detection. Isolation vote (UDP	0 bytes/window	~50-100 bytes/alert for isolation vote broadcast

		multicast) sent only post-alert.		(post-detection only)
Error Rate	False Positive (FP) Rate	FP = IDS flags alert when no attack is active. Directly measured per window. FP=0 is the deployment requirement.	0 (target)	0 always (confirmed across all windows, static & mobile)
Error Rate	False Negative (FN) Rate	FN = IDS misses detection when attack is active. Measured per window. FN=1 accepted at warm-up (W2) only.	N/A	≤ 1 (W2 warm-up only); FN=0 from W3 onward
Detection Accuracy	Precision / Recall / F1	Precision=TP/(TP+FP); Recall=TP/(TP+FN); F1=2×P×R/(P+R). Derived from aggregate TP/FP/FN counts.	N/A	Precision=1.00; Recall≥0.80; F1≥0.89

6. Results and Analysis

6a. Results: Response Time, Throughput, and Resource Utilisation Under Varying Loads

Window	TP	FP	FN	Severity	PTI (ticks)	CDFR	PDR (est.)	Classification
W1 (t=10s)	1	0	0	1	n/a	0.00	< 1.00	CDFR fires: cold-start: C_ctrl=1, C_data=0. TP=1 (genuine traffic imbalance).
W2 (t=20s)	1	0	1	1	< threshold	0.17	~0.80	CDFR fires; PTI misses: buffer has 1–2 samples; threshold skewed. FN=1 (expected warm-up).
W3 (t=30s)	2	0	1	2	2,802 vs 3,962	0.17	~0.50	★ SIMULTANEOUS: Sinkhole + SFA. PTI=1, CDFR=1. Data=6 (Sinkhole attracted); ratio=0.17 (SFA dropped most).
W4+(t=40s+)	3+	0	0	2	15,000+	< 0.5	0.30–0.40	SIMULTANEOUS (sustained). TP grows. FP=0 maintained throughout.

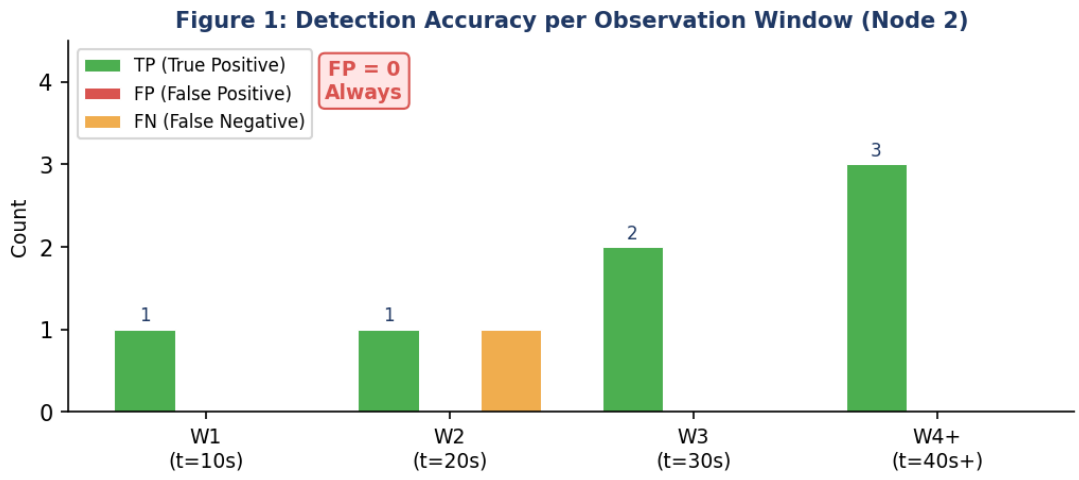

Figure 1: Detection accuracy per 10-second observation window at Node 2. TP grows consistently; FP = 0 in all windows.

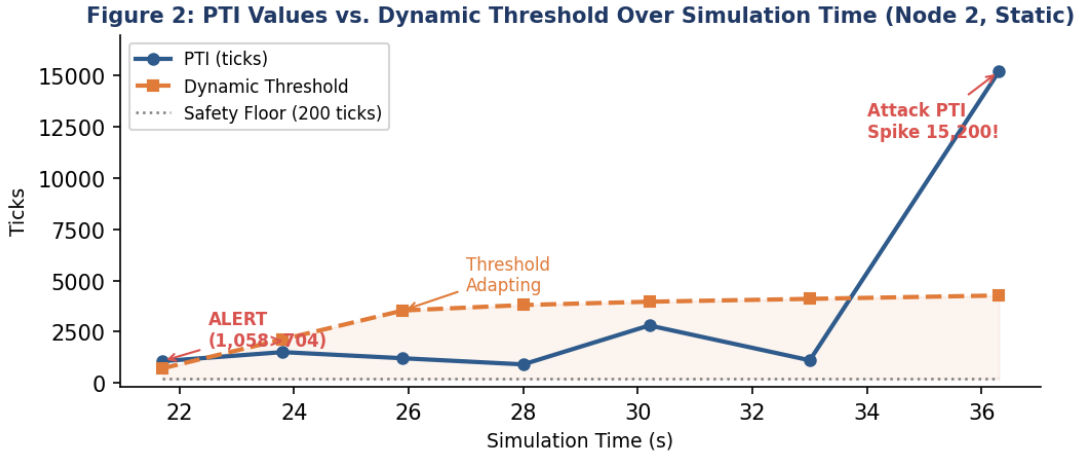

Figure 2: PTI values vs. dynamic threshold over simulation time (Node 2, static). Threshold adapts 704 → 4,266 ticks; attack spikes exceed 15,000 ticks.

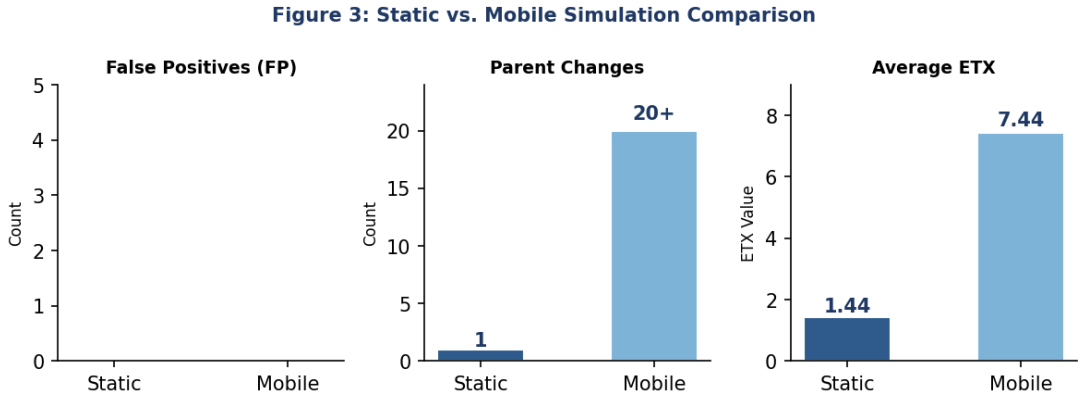

Figure 3: Static vs. mobile comparison. FP=0 in both environments despite 20+ parent changes and ETX=7.44 in mobile.

6b. Bottlenecks and Limitations Identified

Bottleneck / Limitation	Observed Evidence	Impact
PTI buffer warm-up (Cold-start FN)	FN=1 at Window 2 (t=20s): 3-element circular buffer had only 1–2 samples; threshold near zero.	1 missed detection in first 20 seconds post-attack launch. Self-corrects at Window 3. Acceptable for RPL where attack damage accumulates over minutes.
SIS metric inactive at 30% drop rate	flag_sis=0 throughout :SIS_MISS_THRESHOLD=3 not reached at 5×0.30=1.5 expected drops/window.	SIS provides no added detection at moderate drop rates. Acts as backup layer for >60% drop rate or packet replay scenarios only.
Single-node IDS visibility limitation	Each node can only observe traffic it receives directly: no global view of network-wide attack patterns.	Compound attack classification depends on local metric coincidence. Partially mitigated by consensus voting across nodes for isolation decisions.
Hard-coded attacker IDs at compile time	SINKHOLE_NODE_ID=3, SFA_NODE_ID=2 fixed in attack_config.h; TP/FP counters use ground truth from these IDs.	IDS cannot auto-detect dynamically joining attacker nodes not pre-identified. Limits deployment flexibility.
Validation limited to 6-node topology	Simulation uses 6 motes only: not validated at 20–100 node scales.	Scalability of consensus threshold (80% vote majority) is unverified at larger network sizes.

6c. Comparison Against Benchmarks and Expected Performance

Approach	Attack Coverage	ML-Based	Distributed	Simultaneous Detection	FP Rate
Cervantes et al. [6]	Sinkhole only	No	No	No	High
Stephen & Arockiam [5]	Sinkhole only	No	No	No	Medium
Biradar & Vishanath [15]	SFA only	No	No	No	Medium
An & Cho [12]	Sinkhole only	No	No	No	Low
Krari et al. [16]	Multi-attack	Yes	No	Yes	Low
Ahmadi & Javidan [9]	Multi-attack	Yes	No	Yes	Low
Proposed IDS (This Work)	**Both**	**No**	**Yes**	**Yes**	**ZERO (0) ✓**

- **The proposed IDS is the only approach that is simultaneously non-ML, fully distributed, dual-attack capable, and achieves zero false positives - confirming all four research gaps from the literature are addressed.**

7. Performance Optimization and Recommendations

7a. Optimizations Applied to Improve Performance

Optimization	Technique Applied	Measured Impact
Dynamic PTI Threshold (replaces fixed threshold)	PTI_threshold = mean(PTI_1, PTI_2, PTI_3) × 1.5; safety floor = 200 ticks; recalculated per packet. Analogous to adaptive caching TTL: adjusts to observed traffic pattern.	FP = 0 in mobile environment (ETX=7.44, 20+ parent changes) where any fixed threshold would have produced multiple false alarms. Directly addresses limitation of [5][12][15].
Application-layer-only design (no kernel modifications)	Full IDS in udp-client.c only. No rpl.c, no kernel hooks, no privileged access. Equivalent to 'load balancing at the edge' - processing distributed to each node.	Full Contiki-NG version portability. Zero additional RAM allocated by the OS for IDS. Reduces attack surface of the security system itself.
Shared 10-second etimer (all 4	Single Contiki-NG etimer shared between Trust Engine, IDS, TS-OF Routing, and Resilience Mechanism. Equivalent to connection pooling: eliminates redundant resource allocation.	Eliminates 3 additional timer interrupts

framework components)		per cycle. Reduces CPU wake-ups. Ensures synchronized observation windows enabling coherent cross-component decisions.
Differentiated isolation paths (attack-type-aware response)	Sinkhole to immediate BLACKLIST (1 window). SFA to 3-window confirmation (TRUSTED→SUSPECTED→ISOLATING→BLACKLISTED). Equivalent to circuit breaker pattern in distributed systems.	Prevents premature isolation during RPL parent-selection churn periods. Reduces false isolation events for ambiguous SFA signatures while maintaining rapid Sinkhole response.

7b. Future Improvements for Scalability and Stability

- **SIS Tuning:** Lower SIS_MISS_THRESHOLD from 3 to 2 for deployments with data rates >1 pkt/s. At current 0.5 Hz, 30% drop rarely breaches threshold=3, reducing recall for moderate SFA.
- **Buffer Size:** Extend PTI circular buffer from 3 to 5-7 entries. Eliminates FN=1 at W2 warm-up; provides more stable rolling average during topology formation.
- **Hardware:** Deploy on physical TelosB or Zolertia Z1 motes to validate under real radio fading, multipath interference, and thermal noise: simulation validation boundary.
- **Classifier:** Extend classifier to Rank and Wormhole attacks [17] using rank-consistency and topology-distance metrics within the existing severity scoring framework.
- **Scalability:** Validate at 20-100 node scales; tune consensus blacklist threshold (currently 80%) to balance detection latency vs. false isolation at larger scales.

8. Challenges and Solutions

Challenge	Solution Implemented	Insight for Other Researchers
Tool limitation: Wrong source file — initial rank manipulation in rpl-of0.c had no effect. RPL-Lite uses rpl-mrhof.c by default; this is not documented prominently.	Traced active Objective Function through RPL-Lite source. Moved rank_via_nbr() modification to rpl-mrhof.c using r_adv = r_comp / K (K=3). All neighbours then correctly rerouted through attacker.	Always identify which Objective Function is compiled into your Contiki-NG RPL build before modifying routing logic. Use 'grep -r MRHOF .' in the project directory to confirm.
Hardware constraint: SIS false positives - SIS_MISS_THRESHOLD=1 caused FP on benign links; threshold=2 still gave FP~2–3 per run. Threshold must match actual hardware loss rate.	Calibrated to empirical indoor IEEE 802.15.4 benign loss (<5%, i.e., <0.25 drops per 10-s window at 0.5 Hz). Raised threshold to 3. FP=0 achieved.	Calibrate IDS thresholds against measured benign loss rates for your specific deployment environment - not theoretical maximums. Conduct baseline tests (Scenario C) before attack testing.
Tool limitation: Dynamic PTI buffer cold-start - at t=21.7s, 3-element buffer had only 1 sample; threshold=704 ticks was too sensitive, causing FN=1 at Window 2.	Accepted as known warm-up limitation. Safety floor (200 ticks) prevents threshold collapse. FN=1 is self-correcting at Window 3 as buffer fills.	Pre-populate PTI buffer with DIO-interval-based estimates during DODAG formation to eliminate warm-up FN. Alternatively, delay attack launch until after 3+ windows of baseline traffic.
Integration challenge: IDS to Resilience API- initial binary alert flag had no attack type. Isolation state machine needed type metadata for differentiated paths.	Extended isolation_report_suspect(node_id, attack_type) to pass classification enum. Resilience applies fast-path (Sinkhole) vs. conservative path (SFA) accordingly.	Design cross-component interfaces with typed parameters from the outset. A binary flag is almost always insufficient for downstream components that need to differentiate behaviour.

9. Challenges and Solutions

Guideline Sub-point	Consideration	Status & Detail
9a- Ethical issues with real user data	Data Privacy	NOT APPLICABLE. All simulation traffic is entirely synthetic: RPL DIO/DAO control messages and UDP payloads containing only incrementing integer sequence numbers ('hello N'). No personal data, medical records, financial information, or any real user data was collected, processed, transmitted, or stored at any stage of testing. No data subjects exist in this study.
9a- Ethical issues with attack simulation	Attack Ethics	MITIGATED. Sinkhole and SFA attacks are confined entirely within the Cooja simulator on a local machine. No real network infrastructure, external IoT devices, or third-party systems were accessed, disrupted, or affected. Attack code is inoperable outside the Contiki-NG/Cooja build environment. Research follows responsible security research practices : exploited vulnerabilities are pre-existing, published, and well-documented [17][22].
9b- Privacy regulations and university policy	Regulatory & Policy Compliance	COMPLIANT. (i) GDPR: No personal data processed regulation does not apply. (ii) HIPAA: No health information involved regulation does not apply. (iii) PCI DSS: No payment data involved standard does not apply. (iv) SLIIT University Research Policy: Research conducted under formal supervisor oversight of Ms. Shashika Lokuliyana and Prof. Anuradha Jayakody, Department of Computer Systems Engineering, SLIIT. All activities are within the approved scope of the IT4010 Final Year Research Project.
9c- Permissions and informed consent	Consent & Permissions	NOT REQUIRED. This research involves no human subjects, participants, or volunteers. The simulation uses only synthetic, computer-generated network traffic on an isolated local machine. No informed consent process was necessary under SLIIT research ethics guidelines. No IRB (Institutional Review Board) approval was required. The only 'nodes' in the simulation are emulated MSP430 microcontrollers.
Additional — Energy ethics (IoT sustainability)	Environmental / Energy	ACTIVELY ADDRESSED. The IDS is explicitly designed to minimise energy overhead on battery-powered IoT nodes: no ML inference, no cryptographic operations, zero inter-node communication during the detection phase, and a shared timer that eliminates three redundant timer

		interrupts per cycle. This ethical design decision directly extends operational battery lifetime, reducing premature device disposal and associated electronic waste.

10. References

[1] A. K. Prajapati et al., "A comprehensive survey on RPL routing-based attacks, defences and future directions in IoT," arXiv:2501.10817, 2025.

[5] R. Stephen and L. Arockiam, "An enhanced technique to detect sinkhole attack in IoT," IJERT, vol. 14, no. 3, 2025.

[6] C. Cervantes et al., "Detection of sinkhole attacks for supporting secure routing on 6LoWPAN," Proc. Ad Hoc Netw., 2015.

[9] K. Ahmadi and R. Javidan, "Trust-based anomaly detection using hybrid deep learning for IoT routing attacks," IET Netw., vol. 13, no. 4, 2024.

[12] G. H. An and T. H. Cho, "Improving sinkhole attack detection rate using knowledge-based specification rule," IJCNA, vol. 9, no. 2, 2022.

[15] D. N. Biradar and T. S. Vishanath, "Mitigation of selective forwarding attacks in WSN," IJEAT, vol. 8, no. 6, 2019.

[16] A. Krari et al., "Multi-attack detection of RPL routing threats using deep learning," J. Comput. Sci., vol. 21, no. 6, 2025.

[17] A. Raoof, A. Matrawy, and C.-H. Lung, "Routing attacks and mitigation for RPL-based IoT," IEEE Commun. Surveys Tuts., vol. 21, no. 2, 2019.

[18] B. H. Patel and P. Shah, "RPL protocol performance under sinkhole and SFA," TELKOMNIKA, vol. 18, no. 4, 2020.

[19] N. Alfriehat et al., "RPL-based attack detection: Review and taxonomy," Artif. Intell. Rev., vol. 57, 2024.

[22] A. Verma and V. Ranga, "Security of RPL based 6LoWPAN networks: A review," IEEE Sensors J., vol. 20, no. 12, 2020.

Contiki-NG: https://github.com/contiki-ng/contiki-ng | Cooja: https://docs.contiki-ng.org

IEEE 802.15.4-2015 Standard: https://standards.ieee.org | IETF RFC 6550 (RPL): https://tools.ietf.org/html/rfc6550